



Programovacie jazyky

Monády

RNDr. Richard Ostertág, PhD. (ostertag@dcs.fmph.uniba.sk)

16. mája 2017

Katedra informatiky, Fakulta matematiky, fyziky a informatiky
Univerzita Komenského, Bratislava

Monády – viachodnotové funkcie

Motivácia – odmocňovanie nad reálnymi číslami

Majme implementované nasledovné funkcie `rt2r` a `rt3r`, ktoré počítajú pre nezáporné reálne číslo r hodnotu $\sqrt{r}$, resp. $\sqrt[3]{r}$:

```
1 rt2r :: Double -> Double
2 rt2r = sqrt
```

```
1 rt3r :: Double -> Double
2 rt3r r = r ** (1/3)
```

Funkciu `rt6r`, ktorá pre nezáporné reálne číslo r vypočíta $\sqrt[6]{r}$, môžeme definovať jednoduchým zložením funkcií `rt3r` a `rt2r`:

```
1 rt6r :: Double -> Double
2 rt6r = rt3r . rt2r
```

- `rt6r 11.390625` $\rightsquigarrow$ 1.5

Motivácia – odmocňovanie nad komplexnými číslami

Majme funkcie `rt2c` a `rt3c`, ktoré pre ľubovoľné komplexné číslo c vrátia všetky hodnoty x , pre ktoré platí $x^2 = c$, resp. $x^3 = c$:

```
1 rt2c :: Complex Double -> [Complex Double]
2 rt2c c = [x, -x] where x = sqrt c
3
4 rt3c :: Complex Double -> [Complex Double]
5 rt3c c = [x, x * ((-0.5) :+ (-k)), x * ((-0.5) :+ k)]
6   where x = c ** (1/3); k = 0.86602540378443864676
```

Funkciu `rt6c`, ktorá pre ľubovoľné komplexné číslo c vypočíta všetky hodnoty x , pre ktoré platí $x^6 = c$, ale nemôžeme definovať jednoduchým zložením funkcií `rt3c` a `rt2c`:

```
1 rt6c :: Complex Double ->
2                                     [Complex Double]
3 rt6c = rt3c . rt2c -- nie je možné
```

Potrebuje špeciálne skladanie viachodnotových funkcií.

Skladanie viachodnotových funkcií

- aplikovaním vnútornej funkcie $g :: a \rightarrow [b]$ na $x :: a$ získame $[b]$
- na prvky $[b]$ aplikujeme $f :: b \rightarrow [c]$, čím získame $[[c]]$
- zoznam zoznamov vyrovnáme (flatten) funkciou *concat*

```
1 infixr 1 <=<
2 --(.) :: (b -> c) -> (a -> b) -> a -> c
3 (<=<) :: (b -> [c]) -> (a -> [b]) -> a -> [c]
4 f <=< g = \x -> concat $ map f (g x)
```

Teraz už môžeme napísať:

```
1 rt6c :: Complex Double ->
2                                     [Complex Double]
3 rt6c = rt3c <=< rt2c
```

- `rt6c 64` $\rightsquigarrow$ `[2 :+ 0, (-1) :+ (-1.7320508),`
`(-1) :+ 1.7320508, 1 :+ (-1.7320508),`
`(-2) :+ 0, 1 :+ 1.7320508]`

Monády – testovacie výpisy

Motivácia

Majme čisté funkcie `half'` a `double'`:

```
1 half' :: Int -> Int
2 half' x = div x 2
```

```
1 double' :: Int -> Int
2 double' x = x * 2
```

V imperatívnych jazykoch s vedľajšími efektmi nie je problém funkcie rozšíriť o testovacie výpisy. Zachovať ich čistotu môžeme úpravou návratovej hodnoty a tým pádom aj signatúry funkcie:

```
1 half :: Int -> (String, Int)
2 half x = ("Halving " ++ show x, div x 2)
```

```
1 double :: Int -> (String, Int)
2 double x = ("Doubling " ++ show x, x*2)
```

Skladanie funkcií

Funkcie $Int \rightarrow Int$ vieme spolu skladať:

```
1 doublehalf' :: Int -> Int
2 doublehalf' = double' . half'
```

- `doublehalf 9` $\rightsquigarrow$ 8

Ale ich okrášlené (embelished) funkcie $Int \rightarrow (String, Int)$ nie:

```
1 doublehalf :: Int -> (String, Int)
2 doublehalf = double . half -- nedá sa
```

Opäť treba definovať špeciálne skladanie okrášlených funkcií.

Skladanie okrášlených funkcií

Pre skladanie okrášlených funkcií zavedieme operátor `<=<`
(nazývaný aj fish operator):

```
1 infixr 1 <=<
2 --(.) :: (b->      c )->(a->      b )->a->      c
3 (<=<) :: (b->(String,c))->(a->(String,b))->a->(String,c)
4 f <=< g = \x -> let (msg , v ) = g x
5                  (msg' , v') = f v
6                  in (msg ++ "; " ++ msg', v')
```

Teraz už môžeme skladať okrášlené funkcie:

```
1 doublehalf :: Int -> (String, Int)
2 doublehalf = double <=< half
```

- `doublehalf 9` $\rightsquigarrow$ `("Halving 9; Doubling 4",8)`

Monáda

```
1 class Applicative m => Monad m where
2   (>>=) :: m a -> (a -> m b) -> m b
3   (>>)  :: m a -> m b -> m b
4   return :: a -> m a
5   fail   :: String -> m a
6   {-# MINIMAL (>>=) #-}
7   m >> k = m >>= \_ -> k
8   return = pure
9   fail s = errorWithoutStackTrace s
```

(**Either** e), [], **Maybe**, **IO**, ((**->**) a) a **Monoid** a=>((,) a)

- **return** – vloží „čistú“ hodnotu do kontextu
- (>>=) – hod. v kont. odovzdá funkcii vracajúcej hod. v kont.

Implementácia **return** a (>>=) má spĺňať nasledovné identity:

- **return** a >>= k = k a
- m >>= return = m
- m >>= (\x -> k x >>= h) = (m >>= k) >>= h

Maybe, ((,) a) a [] ako monáda

```
1 instance Monad Maybe where
2   (Just x) >>= k = k x
3   Nothing  >>= _ = Nothing
```

```
1 instance Monoid a =>
2   Monad ((,) a) where
3   (u, a) >>= k = case k a of
4     (v, b) -> (u 'mappend' v, b)
```

```
1 instance Monad [] where
2   xs >>= f = [y | x <- xs, y <- f x]
```

do-notácia

- **Just** 3 >>= \x -> **Just** 5 >>= \y -> **Just** \$ x:y:[]

Definícia (do-notácia, bez fail)

$$\text{do } \{e\} = e$$

$$\text{do } \{e; \textit{stmts}\} = e \gg \text{do } \{\textit{stmts}\}$$

$$\text{do } \{p \leftarrow e; \textit{stmts}\} = e \gg= \lambda p \rightarrow \text{do } \{\textit{stmts}\}$$

- **do** {x <- **Just** 3; y <- **Just** 5; return \$ x:y:[]}

```
1  do x <- Just 3
2    y <- Just 5
3    return $ x:y:[]
```

- súvis s množinovou notáciou:
 - [x + y | x <- [1..3], y <- [10, 20]]
 - **do** {x<-[1..3]; y<-[10, 20]; return \$ x+y}

Porovnanie funktorov, aplikatívnych funktora a monád

Monády – if

Pokúsme sa implementovať príkaz if pomocou monád:

```
1 mif :: Monad m=>m Bool->m a->m a->m a
2 mif mb mt me = do
3   b <- mb
4   if b then mt else me
```

alebo ekvivalentný zápis bez použitia do-notácie:

```
1 mif :: Monad m=>m Bool->m a->m a->m a
2 mif mb mt me =
3   mb >>= \b -> if b then mt else me
```

- `mif (return True) (Just 10) Nothing` $\rightsquigarrow$ `Just 10`
- `mif (return True) [1,2] []` $\rightsquigarrow$ `[1,2]`
- `mif (return False) (Just 10) Nothing` $\rightsquigarrow$ `Nothing`
- `mif (return False) [1,2] []` $\rightsquigarrow$ `[]`

Aplikatívne funktory – if

Implementujme príkaz if pomocou aplikatívnych funktorov:

```
1 aif :: Applicative f =>
2     f Bool -> f a -> f a -> f a
3 aif ab at ae = cond<$>ab<*>at<*>ae
4     where cond b t e = if b then t else e
```

alebo ekvivalentný zápis použitím monád:

```
1 xif :: Monad m => m Bool -> m a -> m a -> m a
2 xif mb mt me = do
3     b <- mb; t <- mt; e <- me
4     return $ if b then t else e
```

- `aif (return True) (Just 10) Nothing ~> Nothing`
- `aif (return True) [1,2] [] ~> []`
- `aif (return False) (Just 10) Nothing ~> Nothing`
- `aif (return False) [1,2] [] ~> []`

Iný príklad

```
1 productsA :: [Integer]
2 productsA = (*) <$> [1..3] <*> [1..3]
```

alebo ekvivalentný zápis použitím monád:

```
1 productsM :: [Integer]
2 productsM = do
3   x <- [1..3]
4   y <- [1..3]
5   return (x * y)
```

- `productsA` $\rightsquigarrow$ [1,2,3,2,4,6,3,6,9]
- `productsM` $\rightsquigarrow$ [1,2,3,2,4,6,3,6,9]

Ale pomocou monád vieme spraviť aj

```
1 productsM :: [Integer]
2 productsM = do
3   x <- [1..3]
4   y <- [1..x]
5   return (x * y)
```

- `productsM` $\rightsquigarrow$ `[1,2,4,3,6,9]`

ale ekvivalentný zápis použitím aplikatívnych funktorov neexistuje:

```
1 productsA :: [Integer]
2 productsA = (*) <$> [1..3] <*> [1..???
```

Porovnanie funktorov, aplikatívnych funktorov a monád

Typová trieda	Funkcia	Zdroj	Cieľ
Funktor	$\langle \$ \rangle$	$a \rightarrow b$	$f\ a \rightarrow f\ b$
Aplikatívny funktor	$\langle * \rangle$	$f\ (a \rightarrow b)$	$f\ a \rightarrow f\ b$
Monáda	$\langle = \rangle$	$a \rightarrow f\ b$	$f\ a \rightarrow f\ b$

Smerom od funktorov k monádam získavame silnejšie rozhranie a flexibilnejšiu kontrolu nad výsledkom, ze cenu:

- menších záruk o výsledku,
- menšieho počtu inštancií.

hodnota čistá (strohá) hodnota (napr. a)

kontext zaobaluje čisté hodnoty (napr. $f\ z\ f\ a$)

funktorová hodnota hodnota zabalená v kontexte (napr. $f\ a$)

Kedy čo použiť (funktory, aplikatívne funktory)

Funktor Typ $(a \rightarrow b) \rightarrow (f\ a \rightarrow f\ b)$ funkcie `fmap` `g` `fv` zaručuje, že nemôže zmeniť tvar kontextu nech je funkcia $g : a \rightarrow b$ akákoľvek, pretože tá nevie nič o kontexte. Napríklad:

- `fmap` `g` `[1..10]` vždy vráti desaťprvkový zoznam

Aplikatívny funktor Funkcia $(\langle * \rangle)$ môže meniť tvar kontextu pretože ako funkcie $f (a \rightarrow b)$, tak aj hodnoty $f\ a$ sú vo svojom vlastnom kontexte. Vo vnútri kontextu f sú však čisté funkcie $a \rightarrow b$ a tieto nemôžu meniť kontext. Preto výsledný kontext je úplne určený kontextom argumentov a nazávisí od hodnôt v týchto kontextoch. Napríklad:

- `fs<*>` `[1..10]` vráti zoznam dĺžky `10 * length fs`

Kedy čo použiť (monády)

Monáda Funkcia $=<<$ môže meniť tvar výsledného kontextu podľa vstupných hodnôt, pretože funkcia je typu $a \rightarrow f\ b$ a teda môže vytvoriť vlastný kontext z hodnoty.

- `(\x -> replicate x x) =<< [1..4]`
 $\rightsquigarrow [1,2,2,3,3,3,4,4,4,4]$
- počet prvkov vo výsledku nezýleží len na funkcii na ľavej strane, ale aj od čistých hodnôt uložených v kontexte na pravej strane

Cesta od silnejších k slabším konštrukciám

- Monáda je aplikatívny funktor:
 - `pure = return`
 - `(<*>) = ap`
 - `ap :: (Monad m) => m (a->b) -> m a -> m b`
 - `ap mf mv = do {f<-mf;v<-mv;return(f v)}`
- Aplikatívny funktor je funktor:
 - `fmap f x = pure f <*> x`